The INTERNATIONAL CONFERENCE for HIGH PERFORMANCE COMPUTING NOV 15-20
CHICAGO, IL NETWORKING, STORAGE, & ANALYSIS

SPONSORED BY IEEE COMPUTER SOCIETY TCHPC ACM Association for Computing Machinery sighpc



Auto-Tuning GPU Kernels in Julia

Julia for HPC Tutorial

Floris-Jan Willemsen & Evelyne Ringoot

Overview

Program

- Introduction
- Autotuning basics
- Using optimization algorithms and restrictions
- Energy-efficient autotuning

Practical matters

- Either run the notebooks locally or use the provided AMD GPUs
- Materials can be downloaded at: https://fjwillemsen.com/s/JuliaSC26

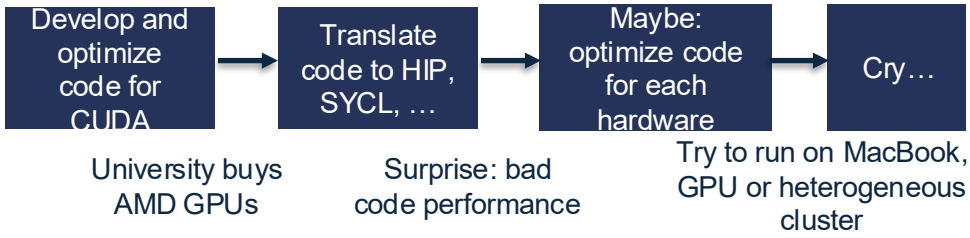


Floris-Jan Willemsen,
Leiden University

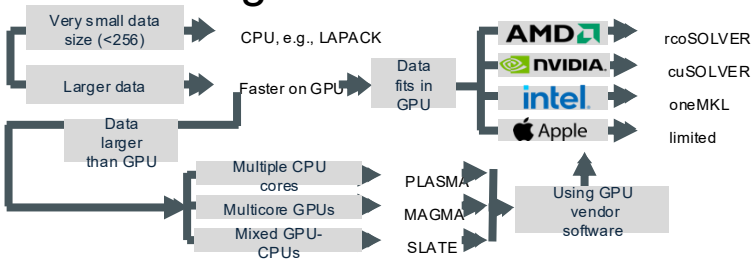


Evelyne Ringoot,
MIT, USA

The portability challenge in HPC

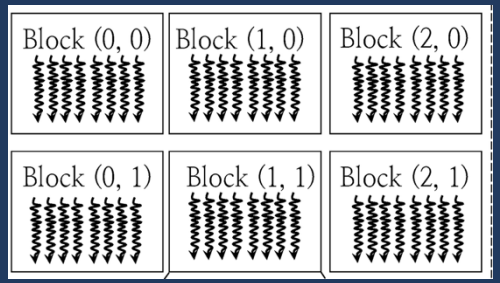


Example: linear algebra libraries



- Porting and re-optimizing code is not sustainable [1]
  - Maintaining spaghetti-code is not sustainable

Driven by different launch models, memory semantics, and compiler intrinsics resulting in different optimal hyperparameter values



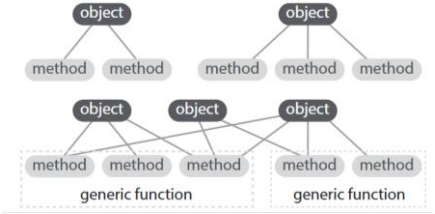
Complexity scales as: libraries × vendors × hardware generations

The Dream: performant, extensible, sustainable

julia : performance through compiler JIT optimization

Multiple-dispatch provides composability

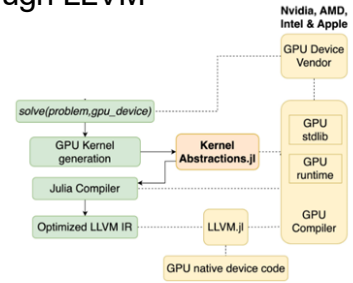
Kernel abstractions: portable code across GPU vendors



(Bezanson, e.a., 2019 [2])

High-level abstractions allow plugging in low-level building blocks and extending to new building blocks seamlessly

Kernels compiled to native code through LLVM



(Utkarsh, e.a., 2023 [3],
library: Churavy, e.a., 2023 [4])

Seamless support for



Wide range of hardware



Novel data precision or mixed precision



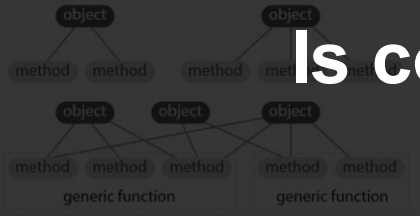
HPC code development scalability

The Dream: performant, extensible, sustainable

julia : performance through compiler JIT optimization

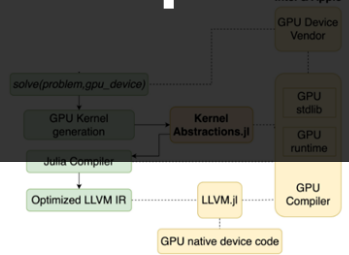
Multiple-dispatch provides composability

Kernel abstractions: portable code across GPU vendors



(Bezanson, e.a., 2019 [2])

Kernels compiled to native code



(Utkarsh, e.a., 2023 [3],
library: Churavy, e.a., 2023 [4])

High-level abstractions allow plugging in low-level building blocks and extending to new building blocks seamlessly

Seamless support for



Wide range of hardware



Novel data precision & mixed precision

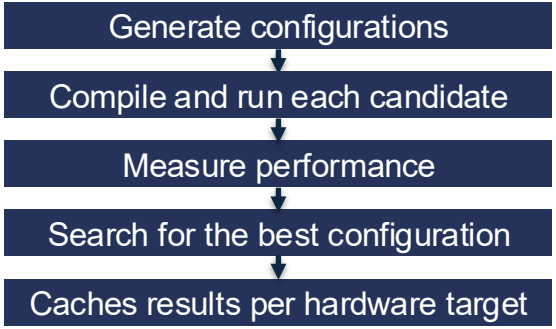


HPC code development scalability

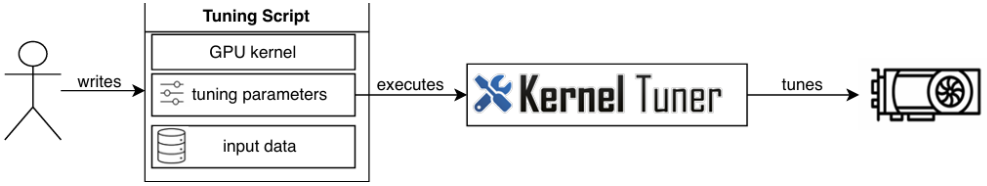
Is compiler optimization sufficient?

Autotuning optimizes parameters by hardware

Autotuning loop



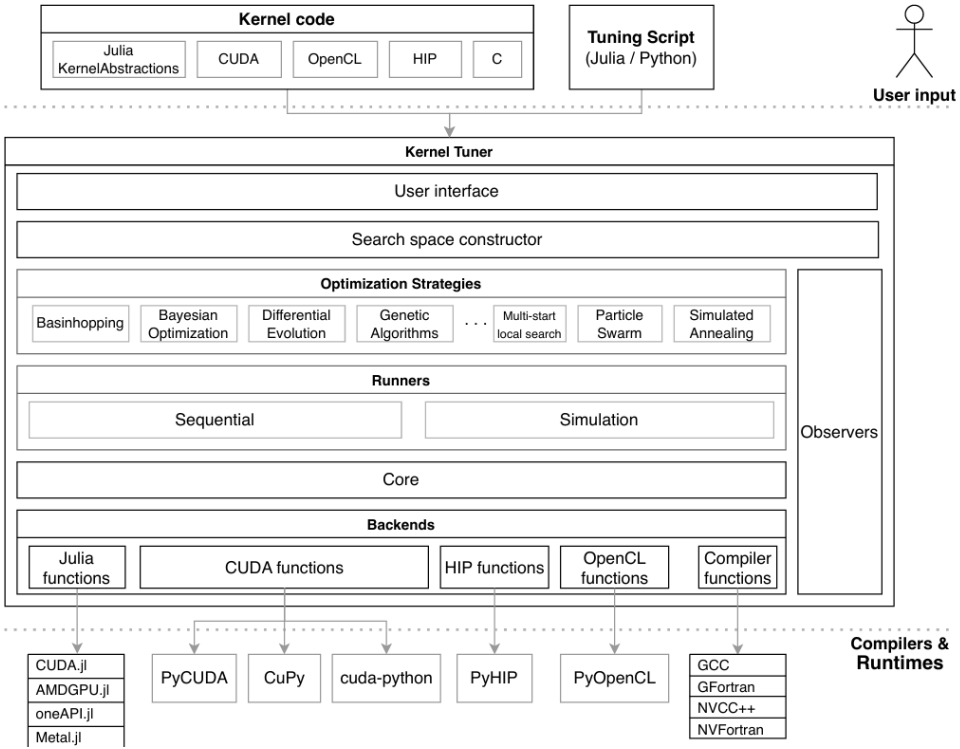
Mature search and optimization infrastructure



Supports:

- CUDA, HIP, OpenCL, C++, Fortran, OpenACC
- 20+ search optimization algorithms and multi-objective Optimization
- Energy measurement of GPU kernels
- Mixed-precision & accuracy tuning

KernelTuner meets KernelAbstractions.jl



Kernel Tuner manages:

- Search space
- Search strategy
- Error handling
- Persistent cache
- Julia compiles and launches GPU kernels
- PythonCall.jl / JuliaCall provide in-process interoperability

Combines Julia portability with Kernel Tuner's mature ecosystem

The INTERNATIONAL CONFERENCE for HIGH PERFORMANCE COMPUTING NOV 15-20
CHICAGO, IL NETWORKING, STORAGE, & ANALYSIS

SPONSORED BY IEEE COMPUTER SOCIETY TCHPC ACM Association for Computing Machinery sighpc



SC26
Chicago, IL | hpc unites.

Auto-Tuning GPU Kernels in Julia

Go to:
fjwillemsen.com/s/JuliaSC26



The INTERNATIONAL CONFERENCE for HIGH PERFORMANCE COMPUTING NOV 15–20
CHICAGO, IL NETWORKING, STORAGE, & ANALYSIS

SPONSORED BY IEEE COMPUTER SOCIETY TCHPC ACM Association for Computing Machinery sighpc



SC26
Chicago, IL | hpc unites.

We care about your experience.

Please take a minute to provide some feedback.

Go to:
fjwillemsen.com/s/JuliaSC26form

Stay Tuned,
Or Contribute.

f.q.willemsen@liacs.leidenuniv.nl
eringoot@mit.edu

Star / Open an issue on:
https://github.com/KernelTuner/KernelTuner.jl



References

[1]: Scalable Delivery of Scalable Libraries and Tools: How ECP Delivered a Software Ecosystem for Exascale and Beyond, Michael A. Heroux

[2] Effective extensible programming: unleashing Julia on GPUs,Besard et al, 2018

[3] Automated translation and accelerated solving of differential equations on multiple GPU platforms, Utkarsh et al, 2023

[4] JuliaGPU/KernelAbstractions.jl , Churavy et al, 2023